

---

## Opticrop: Smart Agricultural Production Optimization Engine

### **Project Description:**

Opticrop is an intelligent, data-driven system designed to revolutionize modern agriculture by leveraging the power of IoT, data analytics, and machine learning. Its primary objective is to assist farmers in making informed, real-time decisions that enhance crop yield, conserve resources, and support sustainable farming practices.

Through a seamless integration of smart sensors, predictive analytics, and intuitive user interfaces, Opticrop continuously monitors critical environmental and crop-related parameters. These insights enable precision farming strategies tailored to the specific needs of each farm—ensuring maximum productivity with minimal input waste.

The system is engineered to tackle real-world agricultural challenges such as unpredictable weather, pest invasions, inconsistent irrigation, and poor crop health visibility—ultimately transforming the way farmers manage their fields.

### **Key Features:**

- **Real-time Data Collection:** Using IoT sensors to monitor soil moisture, temperature, humidity, rainfall, and light levels.
- **Predictive Analytics:** Forecast irrigation needs, detect early pest activity, and predict crop health issues using ML models.
- **Automated Recommendations:** Suggest optimal actions for watering, fertilization, pesticide use, and harvesting.
- **User-Friendly Dashboard:** Visualize data trends, crop conditions, and actionable insights in an intuitive web interface.
- **Sustainability Focused:** Helps in minimizing resource waste, reducing chemical usage, and promoting eco-friendly farming.

### **Use Case Scenarios:**

- **Irrigation Management**

**Challenge:** Inconsistent watering due to changing weather patterns and uneven soil conditions.

**Solution:** Opticrop integrates weather APIs and soil moisture sensors to dynamically optimize irrigation schedules, ensuring crops receive the right amount of water at the right time.

- **Pest Detection and Control**

**Challenge:** Manual identification of pests is time-consuming and often inaccurate.

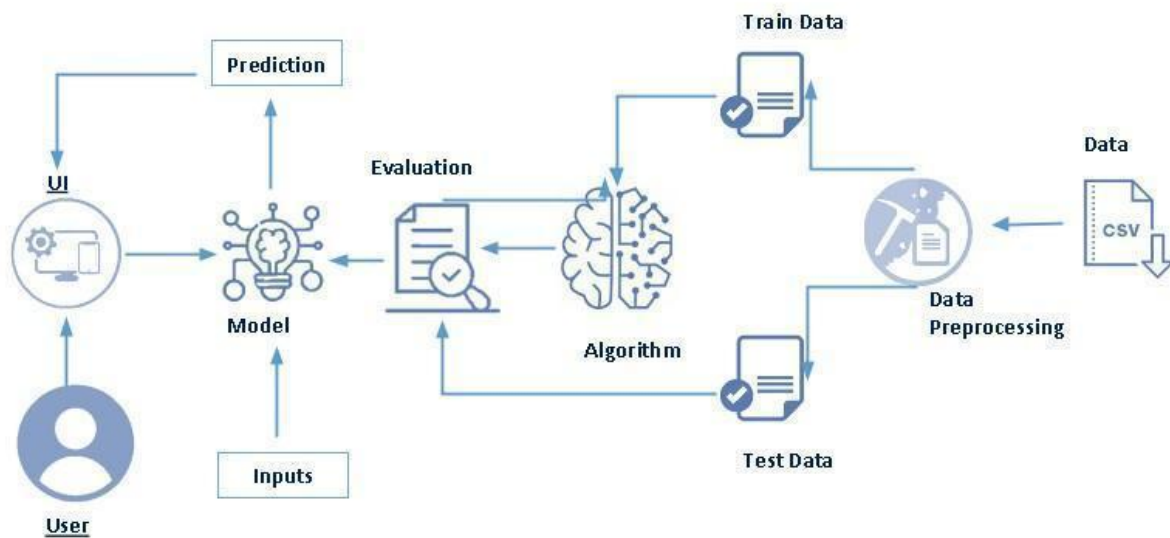
**Solution:** The system uses image-based ML models and field sensors to detect pest

presence and identify the pest type, suggesting targeted biological or chemical treatments.

### Crop Health Monitoring

- Challenge: Monitoring crop health across large farmlands is labour-intensive.  
Solution: Opticrop uses drone and sensor data to assess crop health metrics and detect early symptoms of disease or nutrient deficiencies, enabling timely interventions.

### Technical Architecture:



### Project Flow

- User interacts with the UI to enter the input.
- Entered input is analysed by the model which is integrated.
- Once model analyses the input the prediction is showcased on the UI

To accomplish this, we have to complete all the activities listed below,

- Data collection
  - Collect the dataset or create the dataset
- Visualizing and analysing data
  - Univariate analysis
  - Bivariate analysis
  - Multivariate analysis

- Descriptive analysis
- Data pre-processing
  - Checking for null values
  - Handling outlier
  - Handling categorical data
  - Splitting data into train and test
- Model building
  - Import the model building libraries
  - Initializing the model
  - Training and testing the model
  - Evaluating performance of model
  - Save the model
- Application Building
  - Create an HTML file
  - Build python code

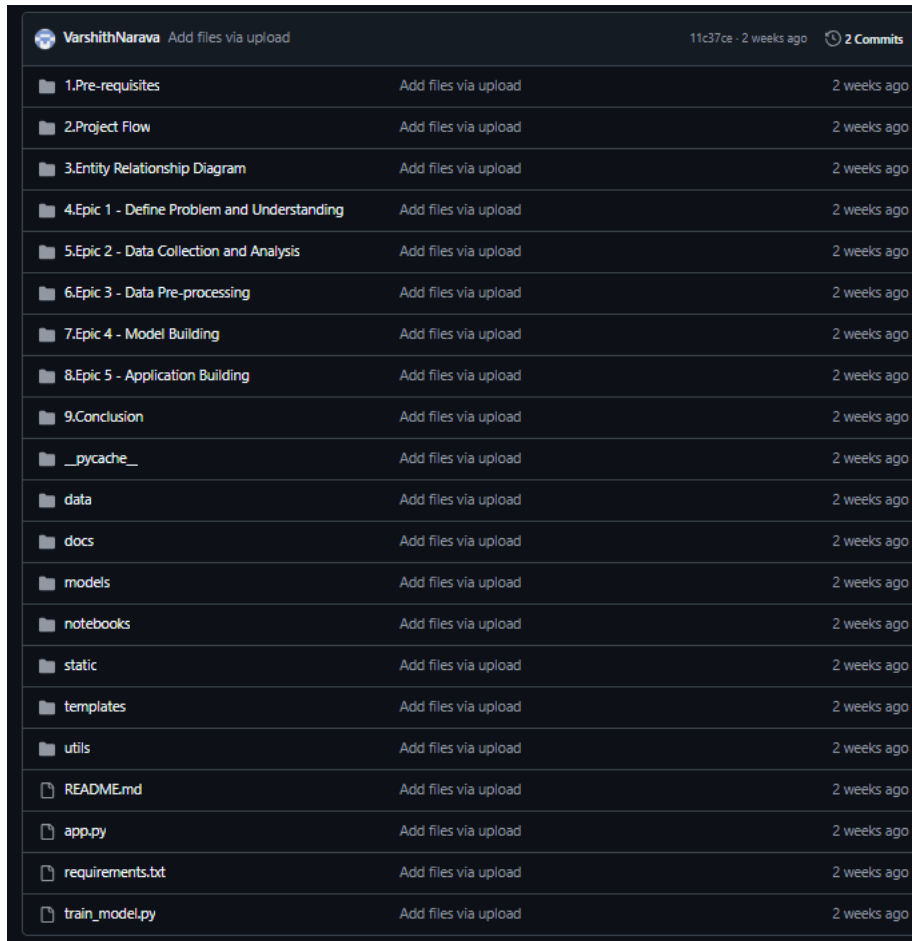
## Prior Knowledge

You must have prior knowledge of following topics to complete this project.

- ML Concepts
  - Supervised learning: <https://www.javatpoint.com/supervised-machine-learning>
  - Unsupervised learning: <https://www.javatpoint.com/unsupervised-machine-learning>
- KMeans: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html> Logistic Regression: <https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-logistic-regression/>
- Evaluation metrics: <https://www.analyticsvidhya.com/blog/2019/08/11-important-model-evaluation-error-metrics/>
- Flask Basics : [https://www.youtube.com/watch?v=Ij4I\\_CvBnt0](https://www.youtube.com/watch?v=Ij4I_CvBnt0)

## Project Structure:

Create the OPTICROP project directory with the following structure:



|                                             |                      |                       |           |
|---------------------------------------------|----------------------|-----------------------|-----------|
| VarshithNarava                              | Add files via upload | 11c37ce · 2 weeks ago | 2 Commits |
| 1.Pre-requisites                            | Add files via upload | 2 weeks ago           |           |
| 2.Project Flow                              | Add files via upload | 2 weeks ago           |           |
| 3.Entity Relationship Diagram               | Add files via upload | 2 weeks ago           |           |
| 4.Epic 1 - Define Problem and Understanding | Add files via upload | 2 weeks ago           |           |
| 5.Epic 2 - Data Collection and Analysis     | Add files via upload | 2 weeks ago           |           |
| 6.Epic 3 - Data Pre-processing              | Add files via upload | 2 weeks ago           |           |
| 7.Epic 4 - Model Building                   | Add files via upload | 2 weeks ago           |           |
| 8.Epic 5 - Application Building             | Add files via upload | 2 weeks ago           |           |
| 9.Conclusion                                | Add files via upload | 2 weeks ago           |           |
| __pycache__                                 | Add files via upload | 2 weeks ago           |           |
| data                                        | Add files via upload | 2 weeks ago           |           |
| docs                                        | Add files via upload | 2 weeks ago           |           |
| models                                      | Add files via upload | 2 weeks ago           |           |
| notebooks                                   | Add files via upload | 2 weeks ago           |           |
| static                                      | Add files via upload | 2 weeks ago           |           |
| templates                                   | Add files via upload | 2 weeks ago           |           |
| utils                                       | Add files via upload | 2 weeks ago           |           |
| README.md                                   | Add files via upload | 2 weeks ago           |           |
| app.py                                      | Add files via upload | 2 weeks ago           |           |
| requirements.txt                            | Add files via upload | 2 weeks ago           |           |
| train_model.py                              | Add files via upload | 2 weeks ago           |           |

## Component Overview:

- **Data/crop\_data.csv:** This file contains tabular data related to crop type, soil conditions, climate metrics, and productivity—used as input for model training and analysis.
- **Python Notebook/Opticrop\_Model\_Development.ipynb:** A Jupyter notebook outlining the step-by-step pipeline including:
  - Data cleaning
  - Feature engineering
  - Model training (e.g., Random Forest, Decision Tree, or Regression models)

- Evaluation and saving the model as model.pkl
- **Templates/**: Contains HTML templates (e.g., index.html, recommendation.html) that form the user interface for interacting with the system.
- **static/**: Holds static content such as CSS styles, JavaScript scripts, and image files used to enrich the frontend experience.
- **app.py**: A Flask server script that:
  - Loads the model.pkl file
  - Handles routing logic
  - Accepts user inputs (e.g., crop type, climate conditions)
  - Returns optimized recommendations for agricultural planning
- **model.pkl**: Serialized machine learning model file used to make predictions within the Flask backend.
- **requirements.txt**: Lists all required Python libraries (e.g., flask, pandas, scikit-learn, joblib, matplotlib) for environment setup.
- **README.md**: Contains an overview of the Opticrop system, setup guide, and usage examples for developers and users.

## Data Collection & Data Pre-processing

### Download the dataset

To begin the project, the required dataset was sourced from a publicly available open-source platform. Numerous such repositories exist, including Kaggle, the UCI Machine Learning Repository, and others.

For this project, we utilized the **Crop\_recommendation.csv** dataset, which was obtained from Kaggle. The dataset can be accessed and downloaded from the following link:

Link: <https://www.kaggle.com/datasets/chitrakumari25/smart-agricultural-production-optimizing-engine>.

Once the dataset was successfully downloaded, we proceeded to perform an in-depth exploration and analysis using various data visualization and analytical techniques to better understand the structure and characteristics of the data.

### Importing the libraries

To begin working with the dataset and building the model, the necessary Python libraries must be imported. These libraries provide essential functionalities for data manipulation, visualization, and machine learning. The most commonly used libraries include:

- Pandas – for data manipulation and analysis
- NumPy – for numerical computations
- Matplotlib / Seaborn – for data visualization
- Scikit-learn – for implementing machine learning algorithms

These libraries can be imported using Python's import statement, as demonstrated in the accompanying implementation.

## IMPORTING THE LIBRARIES

```
In [5]: import pandas as pd
import numpy as np
pd.set_option('max_colwidth', 20)
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 50)
import matplotlib.pyplot as plt
import scipy.stats as stats
import seaborn as sns
%matplotlib inline
plt.rcParams['figure.figsize'] = (12,8)
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
from ipywidgets import interact
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

## Read the Dataset

The dataset can come in various formats such as .csv, .xlsx, .txt, or .json. In this project, the dataset is in .csv format.

We use the **pandas** library to load the dataset into a DataFrame using the `read_csv()` function. This function takes the path to the CSV file as its parameter and returns a structured DataFrame for further processing.

```
In [6]: #Read the dataset
df = pd.read_csv(r"C:\Users\Asus\Desktop\OPTICROP\opticrop\Data\Crop_recommendation.csv")
df.head()
```

```
Out[6]:
```

|   | N  | P  | K  | temperature | humidity  | ph       | rainfall   | label |
|---|----|----|----|-------------|-----------|----------|------------|-------|
| 0 | 90 | 42 | 43 | 20.879744   | 82.002744 | 6.502985 | 202.935536 | rice  |
| 1 | 85 | 58 | 41 | 21.770462   | 80.319644 | 7.038096 | 226.655537 | rice  |
| 2 | 60 | 55 | 44 | 23.004459   | 82.320763 | 7.840207 | 263.964248 | rice  |
| 3 | 74 | 35 | 40 | 26.491096   | 80.158363 | 6.980401 | 242.864034 | rice  |
| 4 | 78 | 42 | 42 | 20.130175   | 81.604873 | 7.628473 | 262.717340 | rice  |

## Data Preprocessing

Before training a machine learning model, it is essential to preprocess the data to enhance its quality and relevance. Raw data often contains inconsistencies, noise, or unwanted patterns that may negatively affect model performance. Preprocessing helps transform the dataset into a clean and usable format.

This step involves the following common operations:

- Handling missing values
- Detecting and addressing outliers
- Managing categorical data

## Handling Missing values

To begin, we analyze the structure and completeness of the dataset:

- `df.shape` is used to determine the number of rows and columns.
- `df.info()` provides data types and memory usage.
- `df.isnull().sum()` helps identify any missing values in each column.

After performing these checks, we observed that the dataset does not contain any null values. Therefore, we can safely skip the missing value treatment step.

### DATA PREPROCESSING

```
In [7]: #Handling missing values

df.shape

df.info()

df.isnull().sum()
```

```
Out[7]: (2200, 8)

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype  
---  --
 0   N               2200 non-null  int64  
 1   P               2200 non-null  int64  
 2   K               2200 non-null  int64  
 3   temperature     2200 non-null  float64 
 4   humidity        2200 non-null  float64 
 5   ph              2200 non-null  float64 
 6   rainfall        2200 non-null  float64 
 7   label           2200 non-null  object  
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB

Out[7]: N          0
       P          0
       K          0
       temperature 0
       humidity    0
       ph          0
       rainfall    0
       label       0
       dtype: int64

In [8]: plt.figure(figsize=(8, 4))
        sns.boxplot(df)
```

## Handling outliers

Outliers are extreme values that can skew the model's performance. To identify them, we utilize **box plots**, which provide a visual summary of the distribution and detect outliers effectively.

In our dataset, the **Potassium (K)** feature exhibited notable outliers, as shown in the boxplot generated using the **Seaborn** library.

To mathematically define outliers, we calculated the **Interquartile Range (IQR)** and determined the **upper** and **lower bounds** as follows:

- **Upper Bound** =  $Q3 + 1.5 \times IQR$
- **Lower Bound** =  $Q1 - 1.5 \times IQR$



To mitigate the effect of these outliers, a **log transformation** technique was applied. Additionally, we created a custom visualization function to display both the distribution and probability plots for the Potassium feature post-transformation.

## Splitting the Dataset

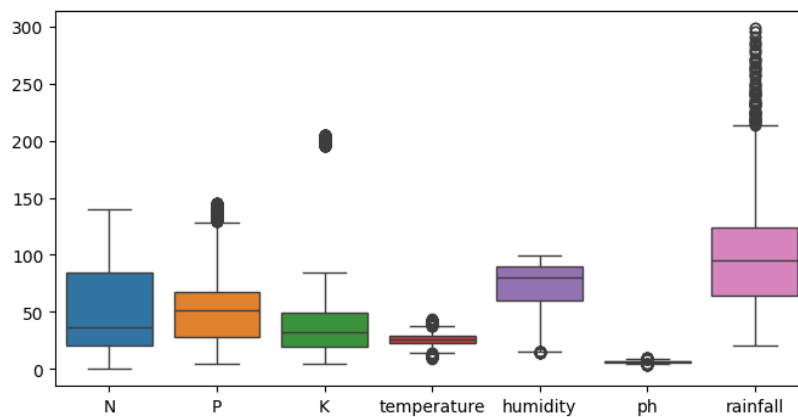
Once preprocessing is complete, we divide the dataset into **features (X)** and **target (y)** variables. The target variable represents the label or output we aim to predict.

- X is constructed by dropping the target column from the DataFrame.
- y contains only the target labels.

We then use the `train_test_split()` function from the **Scikit-learn** library to split the data into **training** and **testing** sets.

Out[8]: <Figure size 800x400 with 0 Axes>

Out[8]: <Axes: >



```
In [9]: Q1 = df['P'].quantile(0.25)
Q3 = df['P'].quantile(0.75)
IQR = Q3 - Q1 #IQR is interquartile range.

filter = (df['P'] >= Q1 - 1.5 * IQR) & (df['P'] <= Q3 + 1.5 * IQR)
df = df.loc[filter]
```

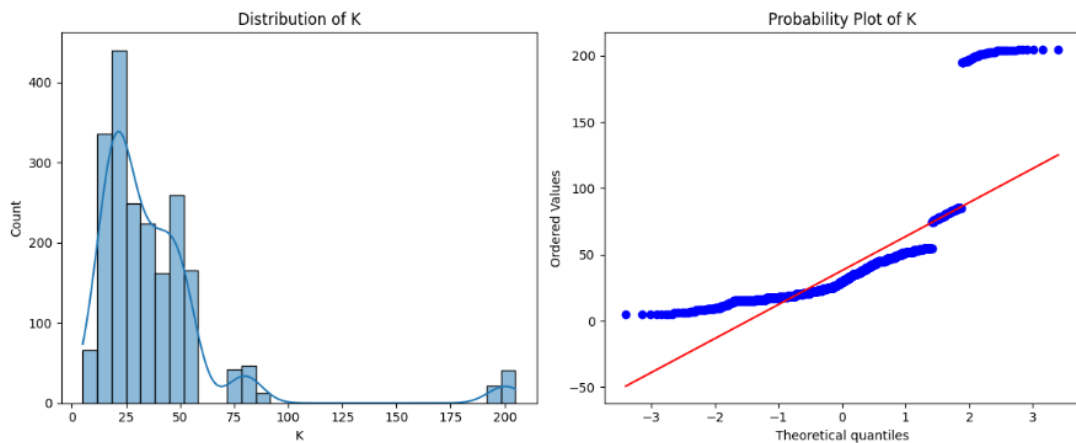
```
In [10]: #changes
def plot_distribution(data, feature):
    plt.figure(figsize=(12, 5))

    # Histogram and KDE
    plt.subplot(1, 2, 1)
    sns.histplot(data[feature], kde=True, bins=30)
    plt.title(f'Distribution of {feature}')

    # Probability plot
    plt.subplot(1, 2, 2)
    stats.probplot(data[feature], dist="norm", plot=plt)
    plt.title(f'Probability Plot of {feature}')

    plt.tight_layout()
    plt.show()
```

```
In [11]: plot_distribution(df, 'K')
```



```
In [12]: df['K_log'] = np.log1p(df['K'])

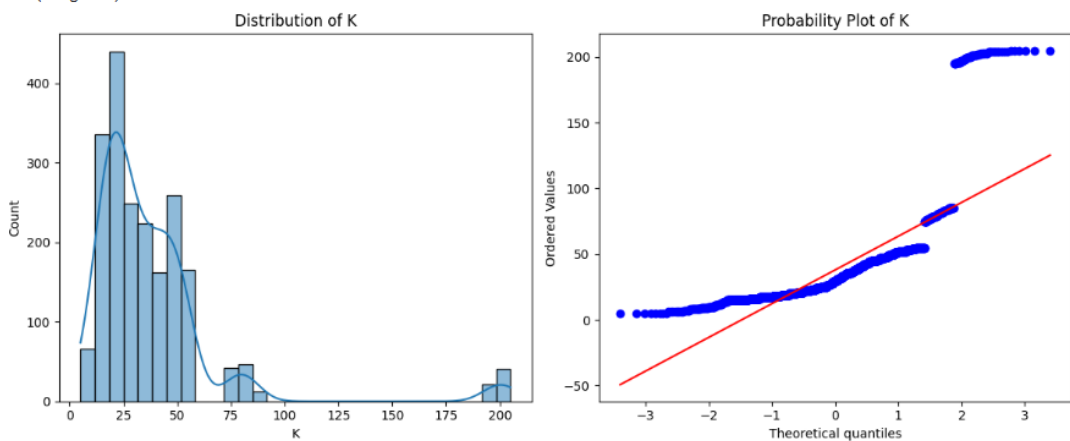
# Optional: Check the result
print(df[['K', 'K_log']].head())
```

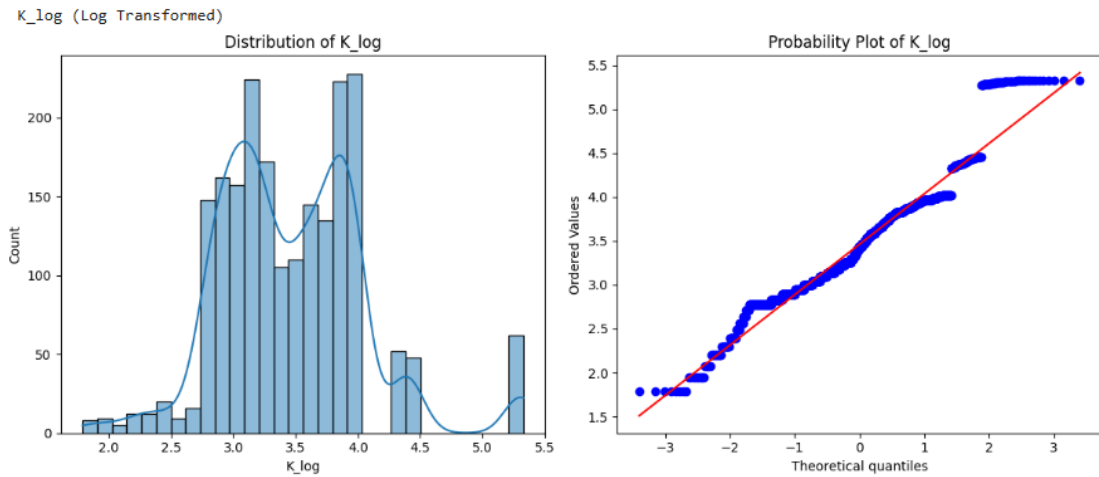
```
   K  K_log
0  43  3.784190
1  41  3.737670
2  44  3.806662
3  40  3.713572
4  42  3.761200
```

```
In [13]: # Original potassium distribution
print("\n K (Original)")
plot_distribution(df, 'K')

# Log-transformed potassium distribution
print("\n K_log (Log Transformed)")
df['K_log'] = np.log1p(df['K'])
plot_distribution(df, 'K_log')
```

K (Original)





```
n [14]: #Extracting Seasonal crops based on conditions
print("Summer crops")
print(df[(df['temperature'] > 30) & (df['humidity'] > 50)]['label'].unique())
print("-----")

print("Winter crops")
print(df[(df['temperature'] < 20) & (df['humidity'] > 30)]['label'].unique())
print("-----")

print("Rainy crops")
print(df[(df['rainfall'] > 200) & (df['humidity'] > 50)]['label'].unique())
print("-----")
```

```
>summer crops
['pigeonpeas' 'mothbeans' 'blackgram' 'mango' 'grapes' 'orange' 'papaya']
-----
Winter crops
['maize' 'pigeonpeas' 'lentil' 'pomegranate' 'grapes' 'orange']
-----
Rainy crops
['rice' 'papaya' 'coconut']
-----
```

```
In [15]: # Features (X): excluded 'label' and used 'K_log' instead of 'K'

X = df.drop(columns=['label', 'K']) # Dropping original 'K', keeping 'K_log'
y = df['label']
X.columns

# Train-test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Printing the shapes
print("The shape of X_train:", X_train.shape)
print("The shape of X_test:", X_test.shape)
print("The shape of y_train:", y_train.shape)
print("The shape of y_test:", y_test.shape)
```

```
Out[15]: Index(['N', 'P', 'temperature', 'humidity', 'ph', 'rainfall', 'K_log'], dtype='object')

The shape of X_train: (1649, 7)
The shape of X_test: (413, 7)
The shape of y_train: (1649,)
The shape of y_test: (413,)
```

```
In [16]: X_train = X_train[['N', 'P', 'K_log', 'temperature', 'humidity', 'ph', 'rainfall']]
X_test = X_test[['N', 'P', 'K_log', 'temperature', 'humidity', 'ph', 'rainfall']]
```

## Data Visualization and Analysis

Data visualization and analysis are essential components in understanding the structure and characteristics of a dataset. By converting raw data into visual formats—such as graphs, plots, and charts—we can simplify complex information and uncover hidden patterns, trends, and anomalies.

This process not only enhances data interpretability but also supports more accurate and informed decision-making. Visualization tools help highlight relationships among features, identify correlations, and detect potential outliers or irregularities.

Combined with statistical analysis, visual exploration enables the extraction of meaningful insights that guide the development of effective models and strategies. Whether in scientific research, business intelligence, or machine learning, this step is vital for unlocking the full potential of data and ensuring a deeper, data-driven understanding.

## VISUALISING AND ANALYSING DATA

### UNIVARIATE ANALYSIS

```
In [17]: features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
titles = [
    'Ratio of Nitrogen', 'Ratio of Phosphorous', 'Ratio of Potassium',
    'Ratio of Temperature', 'Ratio of Humidity', 'Ratio of PH', 'Ratio of Rainfall'
]
colors = ['orange', 'navy', 'pink', 'green', 'blue', 'red', 'yellow']

# Set style
sns.set_style("whitegrid")
plt.figure(figsize=(12, 8))

# Create subplots
for i, feature in enumerate(features):
    plt.subplot(3, 3, i + 1)
    sns.histplot(df[feature], kde=True, color=colors[i])
    plt.title(titles[i])
    plt.xlabel('')
    plt.ylabel('Density')

plt.suptitle("Distribution of agricultural conditions", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

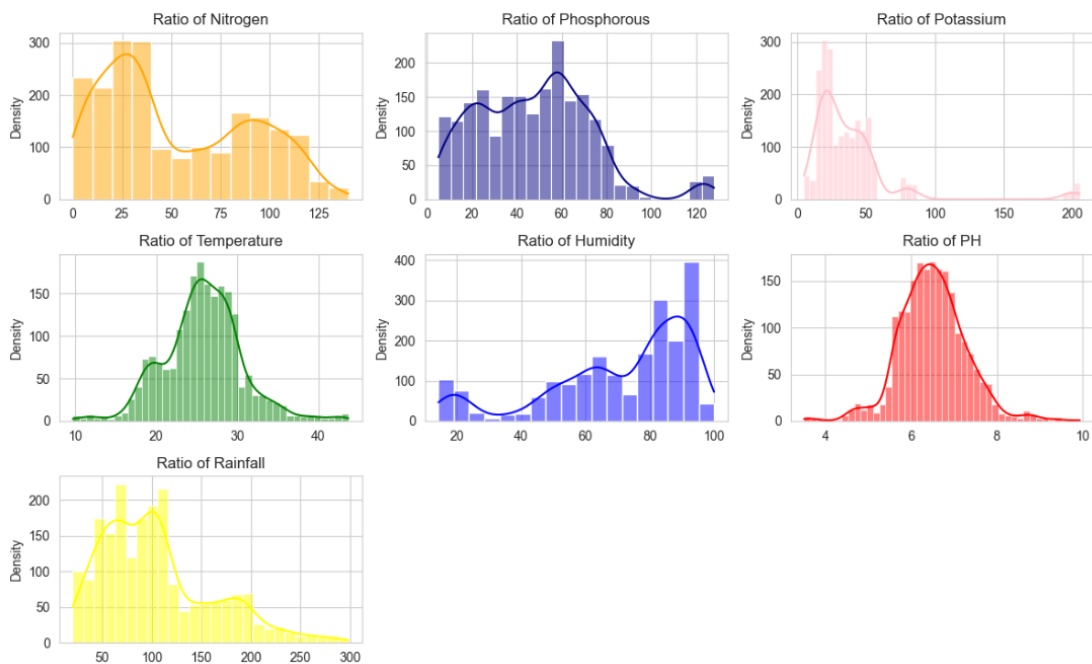
### Univariate Analysis

Univariate analysis involves examining and summarizing the distribution of individual features. This type of analysis helps in understanding the behavior of a single variable without considering the influence of others.

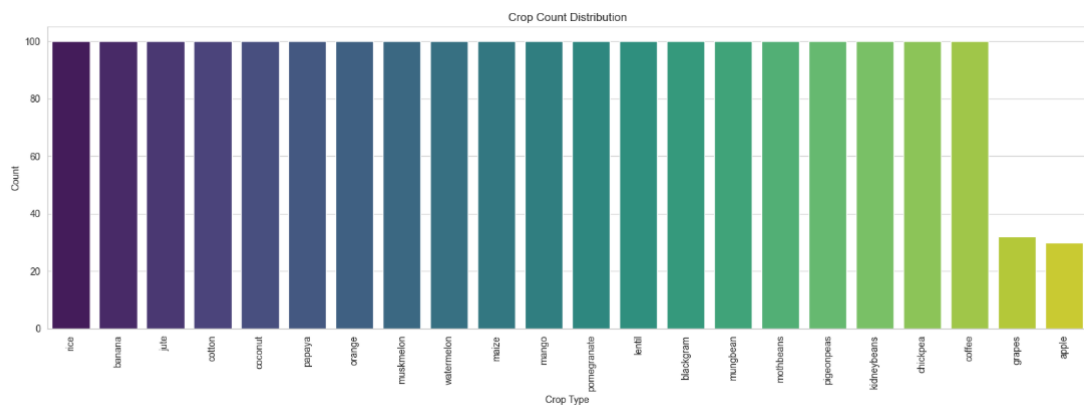
In this project, we used **distplot** and **countplot** from the **Seaborn** library:

- **distplot** provides a visual representation of the distribution of continuous numerical variables, highlighting data concentration and spread.
- **countplot** is used to display the frequency count of categorical variables.

Distribution of agricultural conditions



```
In [18]: #Countplot for Categorical Feature
# =====
plt.figure(figsize=(16, 6))
sns.countplot(data=df, x='label', order=df['label'].value_counts().index, palette='viridis')
plt.xticks(rotation=90)
plt.title('Crop Count Distribution')
plt.xlabel('Crop Type')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```



## Bivariate Analysis

Bivariate analysis is used to explore the relationship between two variables. This helps determine whether changes in one variable are associated with changes in another.

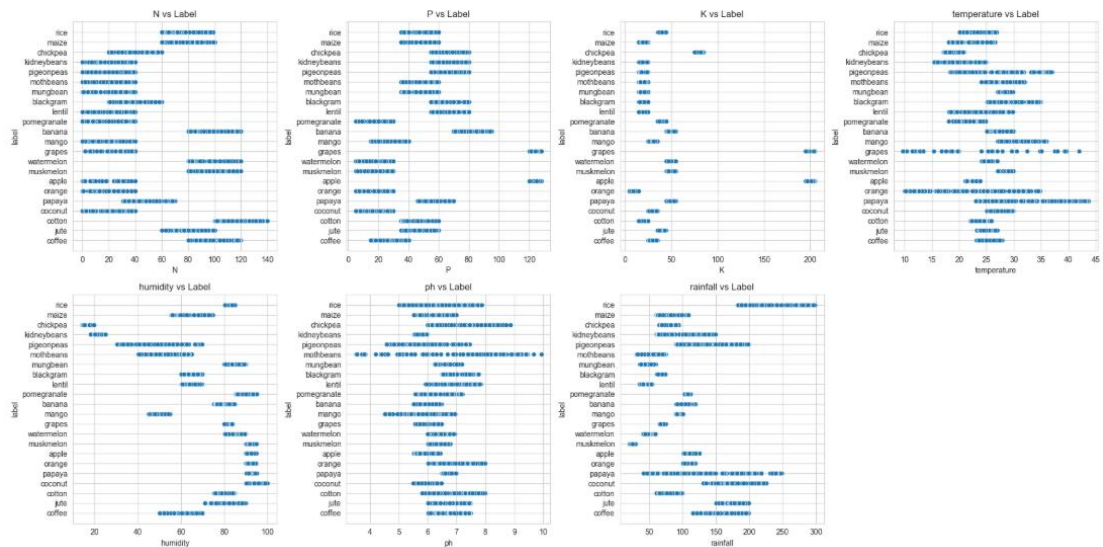
In our analysis, we examined the relationship between **humidity** (a numerical feature) and **label** (the target variable) using suitable plots. This provided insight into how humidity may influence crop recommendations.

## BIVARIATE ANALYSIS

```
In [19]: # Features to plot
features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']

# Create subplots
plt.figure(figsize=(20, 10))
for i, feature in enumerate(features):
    plt.subplot(2, 4, i+1)
    sns.scatterplot(x=df[feature], y=df['label'])
    plt.title(f'{feature} vs Label')
    plt.xlabel(feature)
    plt.ylabel('label')

plt.tight_layout()
plt.show()
```



## Multivariate Analysis

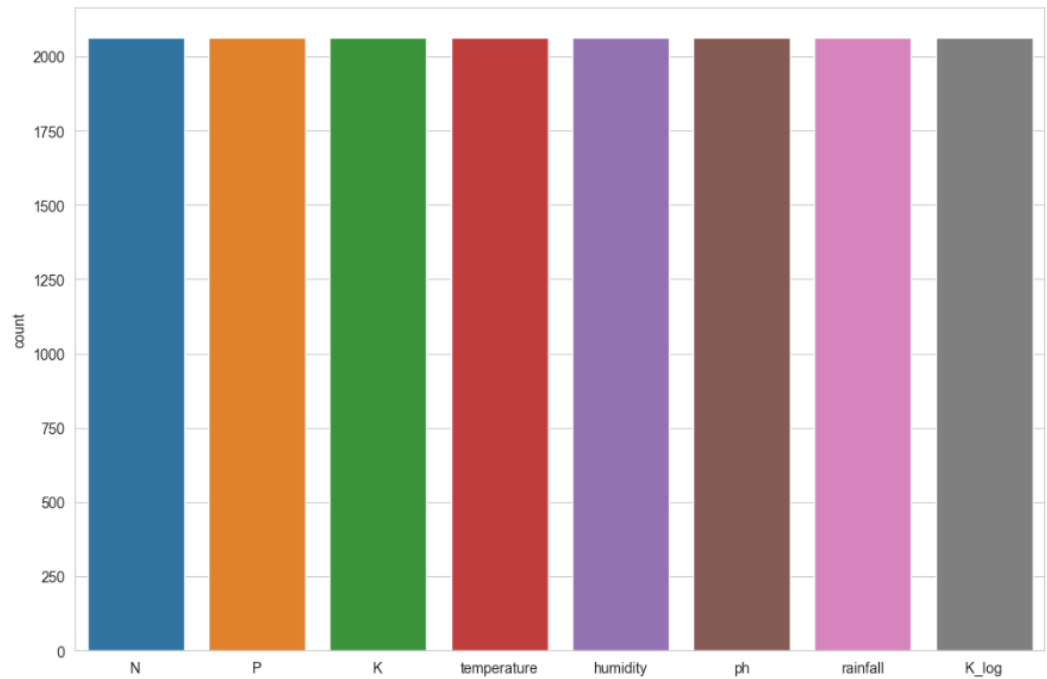
Multivariate analysis investigates the relationships among three or more variables simultaneously. This approach helps in understanding complex interactions within the dataset.

For this purpose, we used **countplot** to visualize how different categories of a target variable relate to multiple independent features. This allows us to assess the combined influence of multiple variables on the output.

MULTIVARIATE ANALYSIS

```
In [20]: sns.countplot(df)

Out[20]: <Axes: ylabel='count'>
```



Descriptive Statistical Analysis

Descriptive analysis involves summarizing the dataset's core characteristics using statistical methods. The **describe()** function in **pandas** offers a convenient summary:

- For **numerical features**, it provides metrics such as mean, standard deviation, minimum, maximum, and percentiles.
- For **categorical features**, it includes values like the most frequent entry, its count, and the number of unique entries.

This statistical overview is crucial for understanding the data's central tendencies and dispersion.

DESCRIPTIVE ANALYSIS

```
In [21]: df.describe()
```

|       | N           | P           | K           | temperature | humidity    | ph          | rainfall    | K_log       |
|-------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| count | 2062.000000 | 2062.000000 | 2062.000000 | 2062.000000 | 2062.000000 | 2062.000000 | 2062.000000 | 2062.000000 |
| mean  | 52.440349   | 47.706596   | 37.996605   | 25.770363   | 70.433297   | 6.502347    | 104.286486  | 3.462439    |
| std   | 37.246143   | 25.479349   | 33.049629   | 4.885812    | 22.569963   | 0.785073    | 56.385635   | 0.587897    |
| min   | 0.000000    | 5.000000    | 5.000000    | 9.724458    | 14.258040   | 3.504752    | 20.211267   | 1.791759    |
| 25%   | 22.000000   | 27.000000   | 20.000000   | 23.080864   | 58.469697   | 6.016597    | 62.940621   | 3.044522    |
| 50%   | 39.000000   | 48.000000   | 30.000000   | 25.768297   | 79.175605   | 6.469677    | 94.772563   | 3.433987    |
| 75%   | 87.000000   | 64.000000   | 46.000000   | 28.614586   | 89.130631   | 6.956328    | 132.787974  | 3.850148    |
| max   | 140.000000  | 128.000000  | 205.000000  | 43.675493   | 99.981876   | 9.935091    | 298.560117  | 5.327876    |

## Model Building

After completing data cleaning and preprocessing, the next step is to develop predictive models. This involves training the dataset using various machine learning algorithms to classify or predict outcomes based on the input features.

In this project, we have implemented and evaluated **four different classification algorithms**. Each model is trained on the pre-processed data, and its performance is assessed using appropriate evaluation metrics.

The model that demonstrates the highest accuracy and generalization capability is selected and **saved for future use**.

This comparative approach ensures that the most suitable algorithm is chosen for reliable and efficient crop recommendation.

## K-Means Clustering

As part of exploratory analysis, we implemented the **K-Means clustering** algorithm to identify natural groupings within the dataset. This unsupervised learning technique groups data points into clusters based on similarity across features, helping uncover hidden patterns among crop types.

## Implementation

The **Elbow Method** was used to determine the optimal number of clusters (k) within the dataset:

- A for loop was used to fit **KMeans** models with cluster numbers ranging from 1 to 10.
- For each iteration, the **Within-Cluster Sum of Squares (WCSS)**—a measure of intra-cluster variance—was calculated and stored.
- A **line plot** was then generated, plotting WCSS against the number of clusters.

This plot helps identify the “elbow point,” where the rate of WCSS reduction sharply decreases. This point indicates the optimal number of clusters to choose.

### MODEL BUILDING

#### KMeans

```
In [22]: # kmeans algorithm is initialized and training data is passed to the model with .fit() function.
# Test data is predicted with .predict() function and saved in new variable.
# For evaluating the model, confusion matrix and classification report is done.

In [23]: # Elbow method used to find out no of clusters and determine the optimum number of clusters within the dataset.
plt.rcParams['figure.figsize'] = (10, 4)

wcscs = []

for i in range(1, 11):
    km = KMeans(n_clusters=i, init="k-means++", max_iter=300, n_init=10, random_state=0)
    km.fit(X)
    wcscs.append(km.inertia_)

plt.plot(range(1, 11), wcscs)
plt.title("The Elbow method", fontsize=20)
plt.xlabel("No of Clusters")
plt.ylabel("WCSS")
plt.show()

Out[23]: KMeans(n_clusters=1, n_init=10, random_state=0)
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

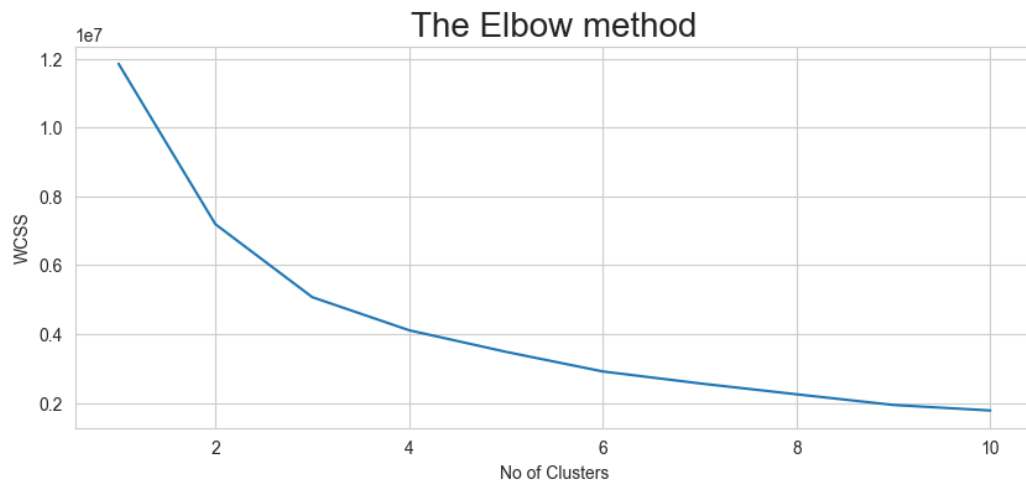
Out[23]: KMeans(n_clusters=2, n_init=10, random_state=0)
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Out[23]: KMeans(n_clusters=3, n_init=10, random_state=0)
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

Out[23]: KMeans(n_clusters=4, n_init=10, random_state=0)
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.
```



```
Out[23]: [matplotlib.lines.Line2D at 0x16bb9f5d550]
Out[23]: Text(0.5, 1.0, 'The Elbow method')
Out[23]: Text(0.5, 0, 'No of Clusters')
Out[23]: Text(0, 0.5, 'WCSS')
```



```
In [24]: km = KMeans(n_clusters=4, init="k-means++", max_iter=300, n_init=10, random_state=0)
y_means = km.fit_predict(X)

a = df['label']
y_means = pd.DataFrame(y_means)
z = pd.concat([y_means, a], axis=1)
z = z.rename(columns={0: 'cluster'})
```

```
print("Lets check the results after applying the K-Means clustering analysis \n")
print("Crops in First cluster:", z[z['cluster'] == 0]['label'].unique())
print("_____ \n")

print("Crops in Second cluster:", z[z['cluster'] == 1]['label'].unique())
print("_____ \n")

print("Crops in Third cluster:", z[z['cluster'] == 2]['label'].unique())
print("_____ \n")

print("Crops in Fourth cluster:", z[z['cluster'] == 3]['label'].unique())
```

Lets check the results after applying the K-Means clustering analysis

Crops in First cluster: ['maize' 'chickpea' 'kidneybeans' 'pigeonpeas' 'mothbeans' 'mungbean'  
'blackgram' 'lentil' nan 'grapes' 'muskmelon' 'apple' 'orange']

Crops in Second cluster: ['maize' 'banana' nan 'grapes' 'watermelon' 'muskmelon' 'orange' 'papaya'  
'coconut' 'cotton' 'jute']

Crops in Third cluster: ['rice' 'pigeonpeas' 'apple' nan 'orange' 'papaya' 'coconut' 'cotton'  
'jute']

Crops in Fourth cluster: ['maize' 'kidneybeans' 'pigeonpeas' 'pomegranate' 'mango' 'muskmelon'  
'apple' nan 'orange' 'papaya' 'cotton' 'jute']

## Logistic Regression Model

To begin the supervised classification task, we implemented the **Logistic Regression** algorithm—one of the most commonly used models for multiclass classification problems.

### Implementation

- The **LogisticRegression** model from `sklearn.linear_model` was initialized.
- The training data was passed to the model using the `.fit()` method to train it on the labeled dataset.

To assess the model's performance:

- A **confusion matrix** was generated to visualize the number of correct and incorrect predictions across all classes.
- A **classification report** was created, providing key metrics such as **precision**, **recall**, **F1-score**, and **accuracy** for each class.

This evaluation helped in understanding how well the Logistic Regression model could classify different crop types based on the input features.

```
In [25]: # Train the logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

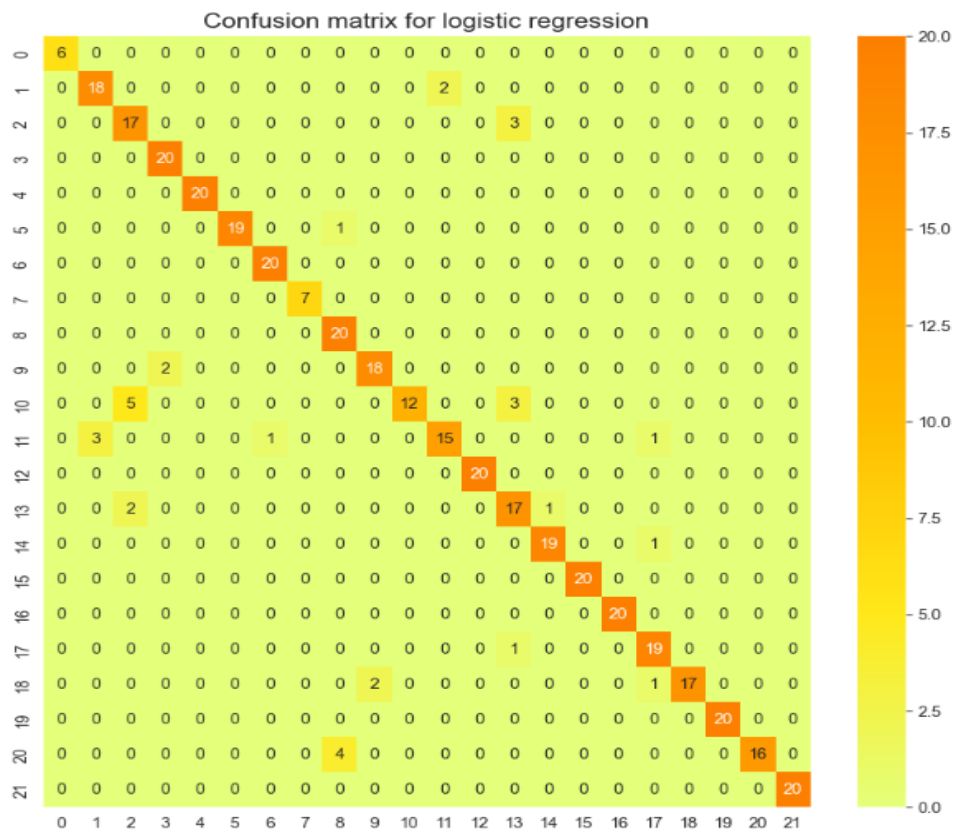
## Evaluating performance of the model and saving the model

```
In [26]: # Let's print the confusion matrix first
plt.rcParams["figure.figsize"] = (10, 10)
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, cmap='wistia')

plt.title("Confusion matrix for logistic regression", fontsize=15)
plt.show()
```

Out[26]: <Axes: >

```
Out[26]: Text(0.5, 1.0, 'Confusion matrix for logistic regression')
```



```
In [27]: cr = classification_report(y_test, y_pred)
print(cr)
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| apple        | 1.00      | 1.00   | 1.00     | 6       |
| banana       | 0.86      | 0.90   | 0.88     | 20      |
| blackgram    | 0.71      | 0.85   | 0.77     | 20      |
| chickpea     | 0.91      | 1.00   | 0.95     | 20      |
| coconut      | 1.00      | 1.00   | 1.00     | 20      |
| coffee       | 1.00      | 0.95   | 0.97     | 20      |
| cotton       | 0.95      | 1.00   | 0.98     | 20      |
| grapes       | 1.00      | 1.00   | 1.00     | 7       |
| jute         | 0.80      | 1.00   | 0.89     | 20      |
| kidneybeans  | 0.90      | 0.90   | 0.90     | 20      |
| lentil       | 1.00      | 0.60   | 0.75     | 20      |
| maize        | 0.88      | 0.75   | 0.81     | 20      |
| mango        | 1.00      | 1.00   | 1.00     | 20      |
| mothbeans    | 0.71      | 0.85   | 0.77     | 20      |
| mungbean     | 0.95      | 0.95   | 0.95     | 20      |
| muskmelon    | 1.00      | 1.00   | 1.00     | 20      |
| orange       | 1.00      | 1.00   | 1.00     | 20      |
| papaya       | 0.86      | 0.95   | 0.90     | 20      |
| pigeonpeas   | 1.00      | 0.85   | 0.92     | 20      |
| pomegranate  | 1.00      | 1.00   | 1.00     | 20      |
| rice         | 1.00      | 0.80   | 0.89     | 20      |
| watermelon   | 1.00      | 1.00   | 1.00     | 20      |
| accuracy     |           |        | 0.92     | 413     |
| macro avg    | 0.93      | 0.93   | 0.92     | 413     |
| weighted avg | 0.93      | 0.92   | 0.92     | 413     |

## Predict the best crop according to given parameters

### Predict the best crop according to given parameters

```
In [28]: prediction = model.predict((np.array([[105, 35, 40, 25, 64, 7, 160]])))
print("The suggested crop for given climatic condition is: ", prediction)
```

The suggested crop for given climatic condition is: ['coffee']

c:\Users\Asus\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names  
warnings.warn(

```
In [29]: !pip install streamlit pyngrok
```

```
In [30]: import pickle

# Save the model in the OPTICROP folder
with open(r'C:\Users\Asus\Desktop\OPTICROP\model.pkl', 'wb') as file:
    pickle.dump(model, file)

print("Model saved in OPTICROP folder")

# Load the model from the same OPTICROP folder
with open(r'C:\Users\Asus\Desktop\OPTICROP\model.pkl', 'rb') as file:
    model = pickle.load(file)

print("Model loaded successfully!")
```

Model saved in OPTICROP folder

Model loaded successfully!

```
In [31]: import pandas as pd

# Sample input (replace with real values if needed)
input_data = pd.DataFrame([[90, 40, 40, 25, 65, 6.5, 120]],
                           columns=['N', 'P', 'K_log', 'temperature', 'humidity', 'ph', 'rainfall'])

# Predict
prediction = model.predict(input_data)

print("Prediction:", prediction[0])
```

Prediction: maize

```
In [ ]:
```

## Application Building

In this section, we focus on developing a web application that integrates the machine learning model trained earlier, allowing users to interact with it through a simple interface.

The web app provides a user interface (UI) where the user can input values such as soil nutrients, temperature, humidity, pH, and rainfall. These inputs are then passed to the saved model, which returns a prediction—displayed directly on the interface.

This section involves the following key tasks:

1. Building HTML Pages:
  - Designed a user-friendly HTML form that collects required feature values.
  - Input fields correspond to the parameters expected by the model.
  - A submit button sends the data to the server for prediction.
2. Developing the Server-Side Script:
  - A backend script (typically written in Python using Flask) receives the user input.
  - The script loads the pre-trained model.
  - It processes the inputs, performs prediction, and returns the result to the frontend.
  - The output (recommended crop) is displayed to the user on the result page.

This approach transforms the machine learning solution into a fully functional application that can be accessed and used by non-technical users, making it more practical and impactful in the agricultural domain.

### Building HTML Pages

As part of the application frontend, three HTML pages were created to provide a structured and user-friendly interface for interacting with the system. These pages were organized under a **Templates** folder, following standard web development practices with Flask.

**The following HTML files were developed:**

1. **home.html**
  - Serves as the **landing page** of the application.
  - Introduces the purpose and features of the crop recommendation system.
  - Includes navigation to other sections such as "About" and "Find Your Crop".
2. **about.html**
  - Provides a brief **overview of the project**, including its objectives, underlying technologies, and benefits.
  - Educates users on how the system works and its relevance to agriculture.
3. **findyourcrop.html**

- Acts as the **core input page** where users can enter values for features such as nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall.
- Contains a **form** that submits data to the backend server for prediction.
- The predicted crop is displayed back to the user on this or a result page.

These HTML files form the **graphical user interface (GUI)** layer of the web application, ensuring a smooth and interactive experience for users accessing the model's functionality.

```

Code Blame 187 lines (175 loc) · 6.89 KB Code 55% faster with GitHub Copilot
Raw Copy Download Edit View

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Find Your Crop - OptiCrop</title>
7   <link rel="stylesheet" href="{{ url_for('static', filename='FindYourCrop.css') }}">
8   <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap" rel="stylesheet">
9   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.0.0/css/all.min.css">
10 </head>
11 <body>
12   <!-- NAVIGATION BAR -->
13   <nav class="navbar">
14     <div class="logo">
15       <i class="fas fa-seedling"></i> OptiCrop
16     </div>
17     <ul class="nav-links">
18       <li><a href="#">Home</a></li>
19       <li><a href="#">About</a></li>
20       <li><a href="/FindYourCrop" class="active">Find Your Crop</a></li>
21     </ul>
22   </nav>
23
24   <div class="overlay">
25     <div class="content">
26       <div class="form-container">
27         <h2>Find Your Perfect Crop</h2>
28         <p class="subtitle">Enter your soil and environmental conditions to get AI-powered crop recommendations</p>
29
30         <form method="POST" action="{{ url_for('FindYourCrop') }}" class="prediction-form">
31           <div class="input-group">
32             <div class="input-field">
33               <i class="fas fa-flask"></i>
34               <input type="text" name="N" placeholder="Nitrogen (N) required">
35             <div class="input-tooltip">Essential nutrient for plant growth</div>
36           </div>

```

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Opti Crop</title>
7   <link rel="stylesheet" href="{{ url_for('static', filename='about.css') }}">
8   <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap" rel="stylesheet">
9   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.0.0/css/all.min.css">
10 </head>
11 <body>
12   <!-- NAVIGATION BAR -->
13   <nav class="navbar">
14     <div class="logo">
15       <i class="fas fa-seedling"></i> OptiCrop
16     </div>
17     <ul class="nav-links">
18       <li><a href="#">Home</a></li>
19       <li><a href="#">About</a></li>
20       <li><a href="/FindYourCrop" class="active">Find Your Crop</a></li>
21     </ul>
22   </nav>
23
24   <div class="overlay">
25     <div class="content">
26       <h2>Opti Crop: Smart Agricultural Optimization Using ML</h2>
27       <p>
28         The project "Opti Crop: Smart Agricultural Production Optimization Engine" aims to develop an advanced
29         software system that utilizes data-driven insights to optimize agricultural production for different crops.
30         By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels,
31         soil temperature, humidity, pH, rainfall, and crop types, Opti Crop seeks to provide intelligent recommendations
32         to farmers for maximizing yields and resource efficiency.
33       </p>
34       <p>
35         The knowledge and insights gained through the Opti Crop project can be utilized by agricultural researchers,
36         policymakers, and stakeholders to advance the understanding of crop-environment interactions and inform the
37         development of evidence-based agricultural strategies.
38       </p>
39     </div>

```

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8" />
5 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 <title>OptiCrop - Smart Agricultural Optimization</title>
7 <link rel="stylesheet" href="{url_for('static', filename='home.css')}" />
8 <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap" rel="stylesheet">
9 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0/css/all.min.css">
10 </head>
11 <body>
12 <!-- NAVIGATION BAR -->
13 <nav class="navbar">
14 <div class="logo">
15 <i class="fas fa-seedling"></i> OptiCrop
16 </div>
17 <ul class="nav-links">
18 <li><a href="/" class="active">Home</a></li>
19 <li><a href="/about">About</a></li>
20 <li><a href="/findYourCrop">Find Your Crop</a></li>
21 </ul>
22 </nav>
23
24 <!-- HERO SECTION -->
25 <div class="overlay">
26 <div class="content">
27 <h2>Welcome to OptiCrop</h2>
28 <p>
29 Revolutionizing agriculture with AI-powered insights. Our smart prediction system helps farmers make data-driven decisions for optimal crop yields and sustainable farming practices.
30 </p>
31 <div class="cta-buttons">
32 <a href="/about" class="glow-button">Learn More</a>
33 <a href="/findYourCrop" class="glow-button secondary">Get Started</a>

```

## Build Python code

### Backend Development – Python with Flask

This section outlines the development of the Python backend using the Flask web framework, which serves as the connection between the trained machine learning model and the HTML user interface.

#### 1. Importing Required Libraries

First, we import all the necessary Python libraries:

- Flask: to create the web server and handle routing.
- render\_template: to load HTML pages.
- request: to handle form submissions from the frontend.
- joblib or pickle: to load the saved machine learning model.

### Loading the Saved Model

The machine learning model, previously trained and saved using joblib or pickle, is loaded into memory to make predictions during user interaction.

### Initializing the Flask App

An instance of the **Flask** class is created. This instance represents our web application.

### Rendering HTML Pages

We define routing rules to connect specific URLs to HTML pages using the Flask `@app.route` decorator.

### Handling Form Submission and Making Predictions

We create a route for handling form submission using a POST method. Input values are collected from the form, passed to the model, and the prediction result is returned to the user.

```

1  import numpy as np
2  from flask import Flask, request, render_template
3  import pickle
4  import os
5
6  app = Flask(__name__)
7
8  # Absolute path to your model file
9  model_path = os.path.join(os.path.dirname(__file__), 'model.pkl')
10
11  # Load the model safely
12  try:
13      with open(model_path, 'rb') as file:
14          model = pickle.load(file)
15  except Exception as e:
16      raise Exception(f"Could not load the model: {e}")
17
18  @app.route('/')
19  def home():
20      return render_template('home.html')
21
22  @app.route('/about')
23  def about():
24      return render_template('about.html')
25
26  @app.route('/FindYourCrop', methods=['GET', 'POST'])

```

```

26  @app.route('/FindYourCrop', methods=['GET', 'POST'])
27  def FindYourCrop():
28      if request.method == 'POST':
29          try:
30              # Match input names from HTML
31              N = float(request.form['N'])
32              P = float(request.form['P'])
33              K = float(request.form['K'])
34              temperature = float(request.form['temperature'])
35              humidity = float(request.form['humidity'])
36              ph = float(request.form['ph'])
37              rainfall = float(request.form['rainfall'])
38
39              # Log transformation
40              K_log = np.log(K + 1)
41
42              # Prediction
43              features = np.array([[N, P, K_log, temperature, humidity, ph, rainfall]])
44              prediction = model.predict(features)
45
46              return render_template('FindYourCrop.html', prediction=prediction[0])
47

```

```

38
39      # Log transformation
40      K_log = np.log(K + 1)
41
42      # Prediction
43      features = np.array([[N, P, K_log, temperature, humidity, ph, rainfall]])
44      prediction = model.predict(features)
45
46      return render_template('FindYourCrop.html', prediction=prediction[0])
47
48      except Exception as e:
49          return render_template('FindYourCrop.html', prediction=f"⚠ Error: {e}")
50
51      return render_template('FindYourCrop.html')
52
53  if __name__ == "__main__":
54      app.run(debug=True)

```

## Running the Application

To run the Opticrop web application, follow these deployment steps using **Visual Studio Code**:

### 1. Launch the Project:

- Open **Visual Studio Code**.
- Import or open the project directory that contains all necessary files and folders, including HTML templates, the trained model, and the main application script (app.py).

### 2. Start the Flask Server:

- Run the app.py file by executing it in the integrated terminal.
- Once the Flask server starts, a local server URL (typically `http://127.0.0.1:5000/`) will be displayed in the terminal.
- Click or paste this URL into your web browser to launch the application.

### 3. Navigating the Application:

- Upon accessing the URL, the application will load the **Home Page**, which serves as the main landing interface.
- From the top navigation menu, clicking on the **"About"** link will redirect you to the **About Page** where details about the project and its objectives are displayed.
- Selecting the **"Recommend"** option from the navigation bar opens the **Crop Recommendation Page**. Here, users can enter values related to soil nutrients, temperature, humidity, pH, and rainfall.

### 4. Generating a Prediction:

- After filling in the required values on the "Soil and Environment Inputs" form, clicking the **"Analyze and Recommend Crop"** button will submit the data to the backend.
- The trained machine learning model processes the input and returns a crop recommendation.
- The predicted result is then displayed on the same page, providing users with real-time, actionable insights.

This marks the successful execution of the Opticrop application, enabling end-to-end interaction from user input to intelligent prediction via a seamless web interface.



HOME PAGE

 **OptiCrop**

[Home](#) [Dashboard](#) [Recommend](#) [Suitability](#) [Crops](#) [Seasonal](#) [Research](#) [About](#)

Smart Agricultural Production Optimization

# OptiCrop

Data-driven crop recommendations powered by machine learning. Optimize yields, improve resource efficiency, and support sustainable farming.

Get Crop Recommendation

Check Crop Suitability

Open Dashboard

22

Crop Types

2,200+

Data Points

7

Environmental Factors

## Three Powerful Scenarios



### Scenario 1: Smart Crop Recommendation

Enter soil NPK levels, temperature, humidity, pH, and rainfall. The ML engine recommends the most suitable crop for maximum yield.

Try it →



### Scenario 2: Suitability Assessment

Evaluate whether your current soil and climate conditions are suitable for a specific crop. Get compatibility scores and productivity insights.

Try it →



### Scenario 3: Research & Policy

Analyze crop-environment relationships, identify patterns, and make data-driven decisions for sustainable agricultural strategies.

Explore →

## Environmental Factors Analyzed

N Nitrogen

P Phosphorus

K Potassium

°C Temperature

% Humidity

pH Soil Acidity

mm Rainfall

ABOUT PAGE

OptiCrop

[Home](#)[Dashboard](#)[Recommend](#)[Suitability](#)[Crops](#)[Seasonal](#)[Research](#)[About](#)

[Home](#) / [About](#)

# About OptiCrop

Smart Agricultural Production Optimization Engine — revolutionizing farming with data-driven intelligence.

## Project Vision

OptiCrop is an advanced software system that utilizes data-driven insights to optimize agricultural production for different crops. By integrating key environmental factors — Nitrogen (N), Phosphorous (P), Potassium (K), soil temperature, humidity, pH, and rainfall — OptiCrop provides intelligent recommendations to farmers for maximizing yields and resource efficiency.

The knowledge gained through OptiCrop can be utilized by agricultural researchers, policymakers, and stakeholders to advance understanding of crop–environment interactions and inform evidence-based agricultural strategies.

## Three Core Scenarios

01

### Smart Crop Recommendation for Farmers

A farmer enters soil and environmental details. The system analyzes the data and recommends the most suitable crop for maximum yield and better farming efficiency.

02

### Crop Suitability and Environmental Assessment

Users evaluate whether current soil and climate conditions are suitable for a particular crop, with compatibility scores and productivity potential.

03

### Agricultural Research and Policy Planning

Researchers and policymakers analyze crop–environment relationships, identify production patterns, and make data-driven decisions for sustainable farming.

## Technology Stack

Flask

Web Framework

Scikit-learn

Machine Learning

Pandas

Data Processing

NumPy

Numerical Computing

Matplotlib

Visualizations

Seaborn

Statistical Charts

SciPy

Scientific Analysis

Random Forest

Classification Model

## CROP RECOMMENDATION PAGE

# Smart Crop Recommendation

Scenario 1 — Enter your soil and environmental conditions to get AI-powered crop recommendations.

### Soil & Environment Inputs

Nitrogen (N)

90

kg/ha ratio

Phosphorus (P)

42

kg/ha ratio

Potassium (K)

43

kg/ha ratio

Temperature

21

°C

Humidity

82

%

pH Level

6.5

soil pH

Rainfall

203

mm

Analyze & Recommend Crop

### Recommendation Results

Rice

Confidence: 98.24%

Based on your soil NPK, climate, and rainfall data, this crop offers the best yield potential.

### Top Alternative Crops

|    |        |        |
|----|--------|--------|
| #1 | Rice   | 98.24% |
| #2 | Jute   | 1.66%  |
| #3 | Papaya | 0.1%   |
| #4 | Apple  | 0%     |
| #5 | Banana | 0%     |

## SUITABLE CROP PAGE

# Crop Suitability Assessment

Scenario 2 — Check whether your conditions are suitable for a specific crop and understand productivity potential.

### Conditions & Target Crop

Select Crop

Banana

▼

Nitrogen (N)

90

kg/ha ratio

Phosphorus (P)

42

kg/ha ratio

Potassium (K)

43

kg/ha ratio

Temperature

21

°C

Humidity

82

%

pH Level

6.5

soil pH

Rainfall

203

mm

Assess Suitability

### Suitability Report

Banana

Not Suitable

31.6%

Overall Score

52.7%

Environmental Fit

0%

ML Confidence

Productivity: Low productivity potential — consider soil amendments or another crop

### Parameter Analysis

|             |     |                     |       |            |
|-------------|-----|---------------------|-------|------------|
| K           | 43  | Ideal: 45.0 – 55.0  | 54.2% | Suboptimal |
| N           | 90  | Ideal: 80.0 – 120.0 | 81.2% | Optimal    |
| P           | 42  | Ideal: 70.0 – 95.0  | 30%   | Suboptimal |
| HUMIDITY    | 82  | Ideal: 75.0 – 85.0  | 86.2% | Optimal    |
| PH          | 6.5 | Ideal: 5.5 – 6.5    | 56.9% | Acceptable |
| RAINFALL    | 203 | Ideal: 90.1 – 119.8 | 30%   | Suboptimal |
| TEMPERATURE |     | Ideal: 25.0 – 29.9  | 30.4% | Suboptimal |